📘 Tutorial Passo a Passo — Prova Prática SAEP (Python + FastAPI)

> TIP
>
> **Meta de 80%+ da Prova com o Mínimo de Código:**
> A prova soma **37 pontos**. Com apenas a Documentação, o Banco SQL, o Login, a Tela Principal e o módulo de **Tutor** (sem nem precisar fazer Pet e Agendamento), o aluno já atinge **30 pontos (81,1% da prova)**! Fazendo os blocos finais de Pet e Agendamento, chega a **100% (37/37 pontos)** em cerca de 1h50min.

🎯 1. A Estratégia para os Alunos

Na prova prática do SAEP o aluno tem **3 horas**, **sem internet**, e o caderno de prova afirma explicitamente: "*o estudante não precisa se preocupar com a estética*". Por isso, nossa arquitetura elimina qualquer complexidade desnecessária:

1. **Zero JavaScript e Zero CSS:** O próprio FastAPI retorna o HTML (`response_class=HTMLResponse`) e recebe os formulários via `<form method="post">` com `Form()` .
2. **Apenas 1 arquivo Python ():** Sem pastas `templates/` , sem arquivos `.js` separados, sem problemas de CORS.
3. **Apenas 3 tabelas (`tutor` , `pet` , `agendamento`):** Como o edital exige que o **Tutor** tenha **CPF e Senha criptografados**, usamos a própria tabela `tutor` para fazer o Login.
4. **Edição na própria linha da tabela (`<tr>`):** Em vez de criar uma tela separada para editar tutor ou atualizar data de agendamento, cada linha da listagem já possui os `<input>` e o botão `Salvar` .

📁 Estrutura Obrigatória de Entrega

```
text
Pasta_Da_Entrega/
 ├── documentacao.txt       (Entrega 1: Requisitos Funcionais e Infraestrutura)
 ├── der.svg / der.pdf      (Entrega 2: Diagrama Entidade-Relacionamento)
 ├── saep_db.sql            (Entrega 3: Script SQL de criação e população)
 └── sistema/               (Entregas 4 a 8: Pasta obrigatória com nome "sistema")
      └── main.py           (Código-fonte completo do sistema)
```

📝 PASSO 1: Documentação de Software — (3 Pontos)

> NOTE
>
> **Itens da Lista de Verificação:**
>
> - **1.1 (Peso 2):** Identificar no mínimo 4 requisitos funcionais.
> - **1.2 (Peso 1):** Identificar todos os requisitos de infraestrutura (SGBD e Linguagem).

Crie o arquivo `documentacao.txt` (o edital aceita `.txt`, `.docx` ou `.pdf`):

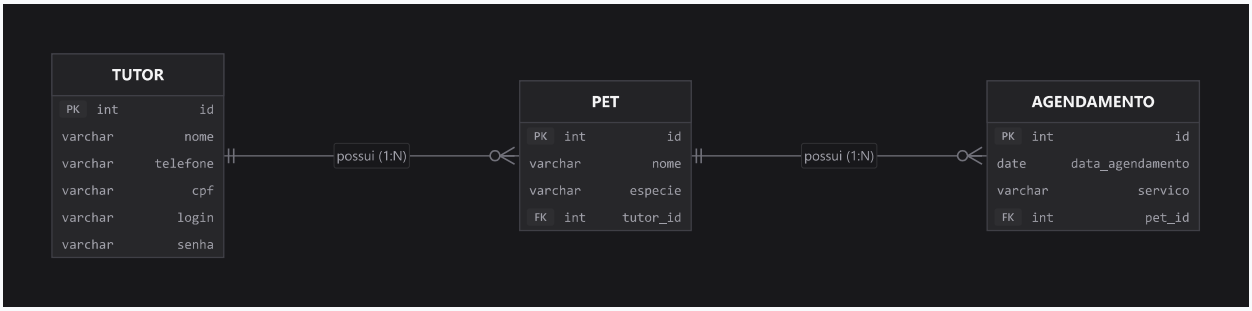
```
text
1.1 REQUISITOS FUNCIONAIS DO SISTEMA (Mínimo 4)
- RF01: O sistema deve permitir o login de usuários validando as credenciais no banco de dados, exibindo mensagem de erro em caso de falha e criando sessão com tempo de expiração.
- RF02: O sistema deve exibir na tela principal o nome do usuário logado na sessão, links para os módulos "Tutor", "Pet" e "Agendamento", e opção para sair do sistema (logout).
- RF03: O sistema deve permitir cadastrar, listar, buscar por termo, editar e excluir tutores, salvando CPF e senha criptografados no banco de dados (LGPD).
- RF04: O sistema deve permitir cadastrar e listar pets associando obrigatoriamente cada pet a um tutor existente.
- RF05: O sistema deve permitir cadastrar agendamentos, atualizar a data de um agendamento e listar os agendamentos ordenados por data exibindo os dados do tutor e do pet.

1.2 LISTA DE REQUISITOS DE INFRAESTRUTURA
- Linguagem de Programação: Python 3 (com framework FastAPI)
- Sistema Gerenciador de Banco de Dados (SGBD): MySQL / SQLite3
- Servidor Web: Uvicorn
- Editor de Código / IDE: VS Code
```

🗂️ PASSO 2: Diagrama Entidade-Relacionamento — (1 Ponto)

> NOTE
>
> **Item da Lista de Verificação:**
>
> - **1.3 (Peso 1):** Gerar o Diagrama Entidade-Relacionamento conforme descrito no caderno.

Desenhe o diagrama com as **3 tabelas** e exporte como imagem ou PDF:



🗄️ PASSO 3: Script do Banco de Dados — (3 Pontos)

> IMPORTANT
>
> **Itens da Lista de Verificação:**
>
> - **2.1 (Peso 2):** Criar as tabelas respeitando a chave estrangeira (`FOREIGN KEY`) de cada relacionamento.
> - **2.2 (Peso 1):** Inserir pelo menos **3 registros em cada uma das tabelas**.

Crie o arquivo `saep_db.sql` sempre na ordem **`tutor`** **->** **`pet`** **->** **`agendamento`** :

```sql
CREATE DATABASE IF NOT EXISTS saep_db;
USE saep_db;

CREATE TABLE IF NOT EXISTS tutor (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    telefone VARCHAR(20) NOT NULL,
    cpf VARCHAR(64) NOT NULL,
    login VARCHAR(50) NOT NULL,
    senha VARCHAR(64) NOT NULL
);

CREATE TABLE IF NOT EXISTS pet (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    especie VARCHAR(50) NOT NULL,
    tutor_id INT NOT NULL,
    FOREIGN KEY (tutor_id) REFERENCES tutor(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS agendamento (
    id INT AUTO_INCREMENT PRIMARY KEY,
    data_agendamento DATE NOT NULL,
    servico VARCHAR(100) NOT NULL,
    pet_id INT NOT NULL,
    FOREIGN KEY (pet_id) REFERENCES pet(id) ON DELETE CASCADE
);

-- 3 Registros por tabela (O hash SHA-256 da senha equivale a '123')
INSERT INTO tutor (nome, telefone, cpf, login, senha) VALUES
('Ana Silva', '11999990001',
'2c624232cdd221771294dfbb310aca000a0df6ac8b66b696d90ef06fdefb64a3', 'ana',
'a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3'),
('Carlos Souza', '11999990002',
'6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b', 'carlos',
'a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3'),
('Mariana Costa', '11999990003',
'd4735e3a265e16eee03f59718b9b5d03019c07d8b6c51f90da3a666eec13ab35', 'mariana',
'a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3');

INSERT INTO pet (nome, especie, tutor_id) VALUES
('Rex', 'Cachorro', 1),
('Mimi', 'Gato', 2),
('Thor', 'Cachorro', 3);
```

```
INSERT INTO agendamento (data_agendamento, servico, pet_id) VALUES
('2026-05-20', 'Consulta Geral', 1),
('2026-03-10', 'Vacinação', 2),
('2025-11-05', 'Banho e Tosa', 3);
```

💻 PASSO 4: Código-Fonte do Sistema — (29 Pontos)

Crie a pasta `sistema` e dentro dela o arquivo `main.py`, seguindo os **6 blocos** abaixo:

Bloco 1: Criptografia LGPD e Conexão com o Banco

A função `criptografar()` tem **2 linhas** e garante os **3 pontos do Item 5.4**:

```
python
import hashlib
import sqlite3
from fastapi import FastAPI, Form, Request
from fastapi.responses import HTMLResponse, RedirectResponse

app = FastAPI()

# ITEM 5.4 (Peso 3): Criptografia de Senha e CPF (LGPD)
def criptografar(texto: str) -> str:
    return hashlib.sha256(texto.encode("utf-8")).hexdigest()

def get_db():
    conn = sqlite3.connect("saep_db.sqlite")
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn

def inicializar_banco():
    conn = get_db()
    conn.executescript("""
        CREATE TABLE IF NOT EXISTS tutor (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            telefone TEXT NOT NULL,
            cpf TEXT NOT NULL,
            login TEXT NOT NULL,
            senha TEXT NOT NULL
        );
        CREATE TABLE IF NOT EXISTS pet (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            especie TEXT NOT NULL,
            tutor_id INTEGER NOT NULL,
            FOREIGN KEY (tutor_id) REFERENCES tutor(id) ON DELETE CASCADE
        );
        CREATE TABLE IF NOT EXISTS agendamento (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            data_agendamento TEXT NOT NULL,
            servico TEXT NOT NULL,
            pet_id INTEGER NOT NULL,
            FOREIGN KEY (pet_id) REFERENCES pet(id) ON DELETE CASCADE
        );
    """)
    if conn.execute("SELECT COUNT(*) FROM tutor").fetchone()[0] == 0:
        senha_padrao = criptografar("123")
```

```
        conn.executemany(
            "INSERT INTO tutor (nome, telefone, cpf, login, senha) VALUES (?, ?, ?, ?, ?)",
            [
                ("Ana Silva", "11999990001", criptografar("11111111111"), "ana",
senha_padrao),
                ("Carlos Souza", "11999990002", criptografar("22222222222"), "carlos",
senha_padrao),
                ("Mariana Costa", "11999990003", criptografar("33333333333"), "mariana",
senha_padrao),
            ],
        )
        conn.executemany(
            "INSERT INTO pet (nome, especie, tutor_id) VALUES (?, ?, ?)",
            [("Rex", "Cachorro", 1), ("Mimi", "Gato", 2), ("Thor", "Cachorro", 3)],
        )
        conn.executemany(
            "INSERT INTO agendamento (data_agendamento, servico, pet_id) VALUES (?, ?, ?)",
            [
                ("2026-05-20", "Consulta Geral", 1),
                ("2026-03-10", "Vacinação", 2),
                ("2025-11-05", "Banho e Tosa", 3),
            ],
        )
        conn.commit()
    conn.close()

inicializar_banco()
```

Bloco 2: Login, Sessão com Expiração e Validação (6 Pontos)

- **Item 3.1 (Peso 1):** Autentica no banco e redireciona para `/principal`.
- **Item 3.2 (Peso 3):** Cria cookie de sessão com `max_age=1800` (30 minutos).
- **Item 3.3 (Peso 2):** Exibe mensagem vermelha de erro se errar a senha.

```
python
@app.get("/", response_class=HTMLResponse)
def tela_login(erro: str = ""):
    msg_erro = f"<p style='color:red;'><b>{erro}</b></p>" if erro else ""
    return f"""
    <h2>Login - Clínica Veterinária</h2>
    {msg_erro}
    <form method="post" action="/login">
        Login: <input name="login" required> <br><br>
        Senha: <input type="password" name="senha" required> <br><br>
        <button type="submit">Entrar</button>
    </form>
    """

@app.post("/login")
def fazer_login(login: str = Form(), senha: str = Form()):
    conn = get_db()
    user = conn.execute(
        "SELECT * FROM tutor WHERE login = ? AND senha = ?",
        (login, criptografar(senha)),
    ).fetchone()
    conn.close()

    if not user:
        return RedirectResponse("/?erro=Login+ou+senha+invalidos!", status_code=303)

    res = RedirectResponse("/principal", status_code=303)
    res.set_cookie(key="usuario_sessao", value=user["nome"], max_age=1800)
    return res

@app.get("/logout")
def logout():
    res = RedirectResponse("/", status_code=303)
    res.delete_cookie("usuario_sessao")
    return res
```

Bloco 3: Tela Principal (4 Pontos)

- **Item 4.1 (Peso 2):** Exibe o nome do usuário logado na sessão.
- **Item 4.2 (Peso 1):** Link para sair do sistema (`/logout`).
- **Item 4.3 (Peso 1):** Links para **Tutor**, **Pet** e **Agendamento**.

```
python
@app.get("/principal", response_class=HTMLResponse)
def tela_principal(request: Request):
    usuario = request.cookies.get("usuario_sessao")
    if not usuario:
        return RedirectResponse("/?erro=Sessao+expirada!", status_code=303)

    return f"""
    <h2>Sistema Clínica Veterinária - Tela Principal</h2>
    <p>Usuário logado: <b>{usuario}</b> | <a href="/logout">Sair do Sistema</a></p>
    <hr>
    <ul>
        <li><a href="/tutores">Gerenciar Tutores</a></li>
        <li><a href="/pets">Gerenciar Pets</a></li>
        <li><a href="/agendamentos">Gerenciar Agendamentos</a></li>
    </ul>
    """
```

Bloco 4: Gerenciamento de Tutores (13 Pontos — 35% da Prova!)

- **Item 5.1 (Peso 2):** Listar e excluir tutores.
- **Item 5.2 (Peso 3):** Busca por termo (`WHERE nome LIKE ?`).
- **Item 5.3 (Peso 2):** Inserir novo tutor.
- **Item 5.4 (Peso 3):** Criptografar CPF e senha com `criptografar()` .
- **Item 5.5 (Peso 3):** Editar tutor existente diretamente na linha da tabela.

```
python
@app.get("/tutores", response_class=HTMLResponse)
def listar_tutores(busca: str = ""):
    conn = get_db()
    if busca:
        tutores = conn.execute("SELECT * FROM tutor WHERE nome LIKE ?", (f"%
{busca}%",)).fetchall()
    else:
        tutores = conn.execute("SELECT * FROM tutor").fetchall()
    conn.close()

    linhas = ""
    for t in tutores:
        linhas += f"""
        <tr>
            <form method="post" action="/tutores/editar">
                <td>{t['id']}<input type="hidden" name="id" value="{t['id']}"></td>
                <td><input name="nome" value="{t['nome']}" required></td>
                <td><input name="telefone" value="{t['telefone']}" required></td>
                <td><input name="login" value="{t['login']}" required></td>
                <td><small>{t['cpf'][:12]}...</small></td>
                <td>
                    <button type="submit">Salvar Edição</button>
                    <a href="/tutores/excluir/{t['id']}">Excluir</a>
                </td>
            </form>
        </tr>
        """

    return f"""
    <p><a href="/principal">Voltar para Principal</a></p>
    <h2>Gerenciamento de Tutores</h2>
    <form method="post" action="/tutores/criar">
        Nome: <input name="nome" required>
        Telefone: <input name="telefone" required>
        CPF: <input name="cpf" required>
        Login: <input name="login" required>
        Senha: <input type="password" name="senha" required>
        <button type="submit">Cadastrar Tutor</button>
    </form>
    <hr>
    <form method="get" action="/tutores">
        Buscar por nome: <input name="busca" value="{busca}">
        <button type="submit">Buscar</button>
        <a href="/tutores">Limpar</a>
    </form>
```

```
    <br>
    <table border="1" cellpadding="5">
        <tr><th>ID</th><th>Nome</th><th>Telefone</th><th>Login</th><th>CPF (Hash LGPD)</th>
<th>Ações</th></tr>
        {linhas}
    </table>
    """

@app.post("/tutores/criar")
def criar_tutor(nome: str = Form(), telefone: str = Form(), cpf: str = Form(), login: str =
Form(), senha: str = Form()):
    conn = get_db()
    conn.execute(
        "INSERT INTO tutor (nome, telefone, cpf, login, senha) VALUES (?, ?, ?, ?, ?)",
        (nome, telefone, criptografar(cpf), login, criptografar(senha)),
    )
    conn.commit()
    conn.close()
    return RedirectResponse("/tutores", status_code=303)

@app.post("/tutores/editar")
def editar_tutor(id: int = Form(), nome: str = Form(), telefone: str = Form(), login: str =
Form()):
    conn = get_db()
    conn.execute("UPDATE tutor SET nome = ?, telefone = ?, login = ? WHERE id = ?", (nome,
telefone, login, id))
    conn.commit()
    conn.close()
    return RedirectResponse("/tutores", status_code=303)

@app.get("/tutores/excluir/{id}")
def excluir_tutor(id: int):
    conn = get_db()
    conn.execute("DELETE FROM tutor WHERE id = ?", (id,))
    conn.commit()
    conn.close()
    return RedirectResponse("/tutores", status_code=303)
```

Bloco 5: Gerenciamento de Pets (3 Pontos)

- **Item 6.1 (Peso 3):** Inserir novo pet associando-o a um tutor via `<select name="tutor_id">`.

```
python
@app.get("/pets", response_class=HTMLResponse)
def listar_pets():
    conn = get_db()
    tutores = conn.execute("SELECT id, nome FROM tutor").fetchall()
    pets = conn.execute("""
        SELECT p.id, p.nome, p.especie, t.nome AS tutor_nome
        FROM pet p JOIN tutor t ON p.tutor_id = t.id
    """).fetchall()
    conn.close()

    opcoes_tutor = "".join([f"<option value='{t['id']}'>{t['nome']}</option>" for t in
tutores])
    linhas = "".join([f"<tr><td>{p['id']}</td><td>{p['nome']}</td><td>{p['especie']}</td>
<td>{p['tutor_nome']}</td></tr>" for p in pets])

    return f"""
    <p><a href="/principal">Voltar para Principal</a></p>
    <h2>Gerenciamento de Pets</h2>
    <form method="post" action="/pets/criar">
        Nome do Pet: <input name="nome" required>
        Espécie: <input name="especie" required>
        Tutor: <select name="tutor_id" required>{opcoes_tutor}</select>
        <button type="submit">Cadastrar Pet</button>
    </form>
    <hr>
    <table border="1" cellpadding="5">
        <tr><th>ID</th><th>Nome do Pet</th><th>Espécie</th><th>Tutor Associado</th></tr>
        {linhas}
    </table>
    """

@app.post("/pets/criar")
def criar_pet(nome: str = Form(), especie: str = Form(), tutor_id: int = Form()):
    conn = get_db()
    conn.execute("INSERT INTO pet (nome, especie, tutor_id) VALUES (?, ?, ?)", (nome,
especie, tutor_id))
    conn.commit()
    conn.close()
    return RedirectResponse("/pets", status_code=303)
```

Bloco 6: Gerenciamento de Agendamentos (5 Pontos)

- **Item 7.1 (Peso 3):** Listar agendamentos trazendo dados do tutor e do pet, ordenados por data.

- **Item 7.2 (Peso 2):** Atualizar a data de um agendamento existente.

```python
@app.get("/agendamentos", response_class=HTMLResponse)
def listar_agendamentos():
    conn = get_db()
    pets = conn.execute("SELECT id, nome FROM pet").fetchall()
    # Prova 1 (Dia): ORDER BY strftime('%d', a.data_agendamento) ASC
    # Prova 2 (Mês): ORDER BY strftime('%m', a.data_agendamento) ASC
    # Prova 3 (Ano): ORDER BY strftime('%Y', a.data_agendamento) ASC
    agendamentos = conn.execute("""
        SELECT a.id, a.data_agendamento, a.servico,
               p.nome AS pet_nome, p.especie AS pet_especie,
               t.nome AS tutor_nome, t.telefone AS tutor_telefone
        FROM agendamento a
        JOIN pet p ON a.pet_id = p.id
        JOIN tutor t ON p.tutor_id = t.id
        ORDER BY a.data_agendamento ASC
    """).fetchall()
    conn.close()

    opcoes_pet = "".join([f"<option value='{p['id']}'>{p['nome']}</option>" for p in pets])
    linhas = ""
    for a in agendamentos:
        linhas += f"""
        <tr>
            <td>{a['id']}</td>
            <td>
                <form method="post" action="/agendamentos/atualizar" style="margin:0;">
                    <input type="hidden" name="id" value="{a['id']}">
                    <input type="date" name="data_agendamento" value="
{a['data_agendamento']}" required>
                    <button type="submit">Atualizar Data</button>
                </form>
            </td>
            <td>{a['servico']}</td>
            <td>{a['pet_nome']} ({a['pet_especie']})</td>
            <td>{a['tutor_nome']} - Tel: {a['tutor_telefone']}</td>
        </tr>
        """

    return f"""
    <p><a href="/principal">Voltar para Principal</a></p>
    <h2>Gerenciamento de Agendamentos</h2>
    <form method="post" action="/agendamentos/criar">
        Data: <input type="date" name="data_agendamento" required>
        Serviço: <input name="servico" required>
        Pet: <select name="pet_id" required>{opcoes_pet}</select>
```

```
        <button type="submit">Agendar</button>
    </form>
    <hr>
    <table border="1" cellpadding="5">
        <tr><th>ID</th><th>Data (Ordenada)</th><th>Serviço</th><th>Dados do Pet</th>
<th>Dados do Tutor</th></tr>
        {linhas}
    </table>
    """

@app.post("/agendamentos/criar")
def criar_agendamento(data_agendamento: str = Form(), servico: str = Form(), pet_id: int =
Form()):
    conn = get_db()
    conn.execute("INSERT INTO agendamento (data_agendamento, servico, pet_id) VALUES (?, ?,
?)", (data_agendamento, servico, pet_id))
    conn.commit()
    conn.close()
    return RedirectResponse("/agendamentos", status_code=303)

@app.post("/agendamentos/atualizar")
def atualizar_data_agendamento(id: int = Form(), data_agendamento: str = Form()):
    conn = get_db()
    conn.execute("UPDATE agendamento SET data_agendamento = ? WHERE id = ?",
(data_agendamento, id))
    conn.commit()
    conn.close()
    return RedirectResponse("/agendamentos", status_code=303)
```

📊 Tabela de Pontuação Acumulada (Guia de Prioridade na Prova)

Ordem de Execução	Peso	Acumulado	% da Prova
1. Documentação (`documentacao.txt`) + DER (`der.svg`)	4 pts	4 pts	10,8%
2. Banco de Dados (`saep_db.sql`)	3 pts	7 pts	18,9%
3. Login + Sessão (`max_age`) + Validação de Erro	6 pts	13 pts	35,1%
4. Tela Principal (Nome da sessão + Menu + Logout)	4 pts	17 pts	45,9%
5. Gerenciamento de Tutor (CRUD + Busca + `hashlib`)	13 pts	**30 pts**	**81,1% (Meta 80% batida!)**
6. Gerenciamento de Pets (Cadastro com `<select>`)	3 pts	33 pts	89,2%
7. Gerenciamento de Agendamentos (Listar ordenado + Mudar data)	5 pts	**37 pts**	**100,0%**